* Alguns dos Exercícios foram retirados/adaptados do curso Python Programming MOOC 2026

1) O exercício 3 dos slides sobre Hierarquia de Classes.
2) O exercício 4 dos slides sobre Hierarquia de Classes.
3) O exercício 5 dos slides sobre Hierarquia de Classes.

4) Crie uma classe chamada `Laptop` que herda da classe `Computer`.

   A classe `Laptop` deve ter um método construtor `__init__(self, model: str, speed: int, weight: int)` que chama o construtor da classe pai `Computer`. O parâmetro `weight` (peso) deve ser armazenado em um atributo privado, e a classe deve fornecer uma propriedade getter `@property` chamada `weight` para acessá-lo.

   A classe também deve ter um método `__str__`, que retorna uma representação em string do objeto no seguinte formato:

   ```
   Notebook Pro, 1500 MHz, 2 kg
   ```

   Exemplo de uso:

   ```
   laptop = Laptop("Notebook Pro", 1500, 2)
   print(laptop)
   ```

   Exemplo de saída:

   ```
   Notebook Pro, 1500 MHz, 2 kg
   ```

5) Defina uma classe chamada `Rectangle` (Retângulo) com atributos width e height (largura e altura), além de um método que retorna sua area e o seu __str__. Construa também a classe `Square` (Quadrado) que herda da classe `Rectangle` e ajuste seus atributos e métodos.

6) Classes de Pagamento:

   - Modele a superclasse `Pagamento` com um método `processar()`.
   - Implemente as subclasses `PagamentoPix` e `PagamentoCartao` herdando da classe base.
   - Garanta que cada forma de pagamento implemente sua própria lógica/mensagem no método `processar()`.
   - Simule um carrinho de compras criando uma lista de pagamentos mistos e processe todos eles iterativamente em um único loop `for`.

7) Você foi contratado por uma gráfica digital para criar o protótipo do módulo de geração de cartões comemorativos virtuais. Sua tarefa é implementar a estrutura de classes para os cartões:
   - Modele uma classe base `CartaoMensagem` com os campos `destinatario` e `remetente`, e uma assinatura de método `retornar_mensagem()`.
   - Estenda a classe base criando cartões específicos para **Dia dos Namorados**, **Natal** e **Aniversário**, garantindo que cada um imprima a felicitação adequada juntamente com a assinatura do remetente e o nome do destinatário.
   - Monte uma rotina interativa que peça os dados para o usuário via terminal, crie os três cartões e exiba todas as mensagens geradas de forma dinâmica.

8) Jogo Pedra, Papel, Tesoura :Você irá criar o bastante conhecido jogo de “pedra, papel, tesoura”, onde um jogador humano irá jogar rodadas contra o computador e ao final será mostrado o resultado do placar de vitórias pra cada jogador. Para isso você usará POO, com herança e polimorfismo, através das seguintes classes:

   Classe Jogador:
   a) Atributos:
      i) nome (string)
      ii) escolha (inicialmente vazio (None), mas será “pedra”, “papel” ou “tesoura”)
   b) Métodos
      i) __init__(self): O construtor da classe que inicializa os atributos.
      ii) EscolherJogada(): Definirá a escolha de cada jogador. (Método abstrato)

   Classe Humano: Herda de Jogador
   c) Método:
      i) EscolherJogada(): Questiona o usuário qual a sua jogada: pedra, papel ou tesoura (apenas essas 3 opções, pedir pra repetir caso entrada diferente) e registra no atributo de escolha.

   Classe Computador: Herda de Jogador
   d) Método:
      i) EscolherJogada(): De forma aleatória escolhe uma das três opções pedra, papel ou tesoura e registra no atributo de escolha.

   Classe Jogo:
   e) Atributos:
      i) Dois Jogadores (Computador ou Humano)
      ii) Placar (dicionário)
   f) Métodos
      i) __init__(self): O construtor da classe que inicializa os atributos.
      ii) JogarPartida(): Em loop até que se deseje parar o jogo, manda que cada jogador realize escolhas de cada jogador e mostra os resultados de cada rodada, ao fim mostra o placar final. (Há polimorfismo sendo usado aqui?)
      iii) DefineVencedor(): Define quem venceu a rodada